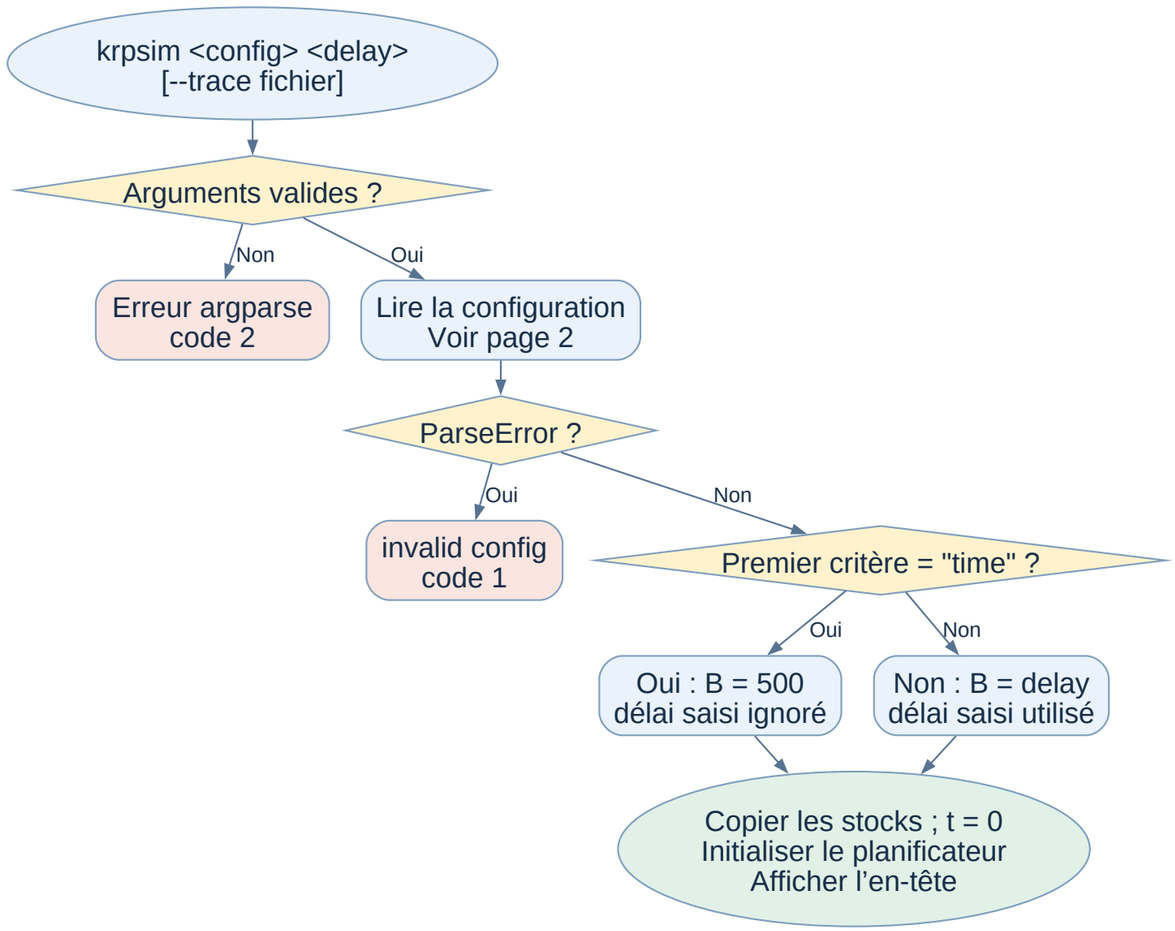
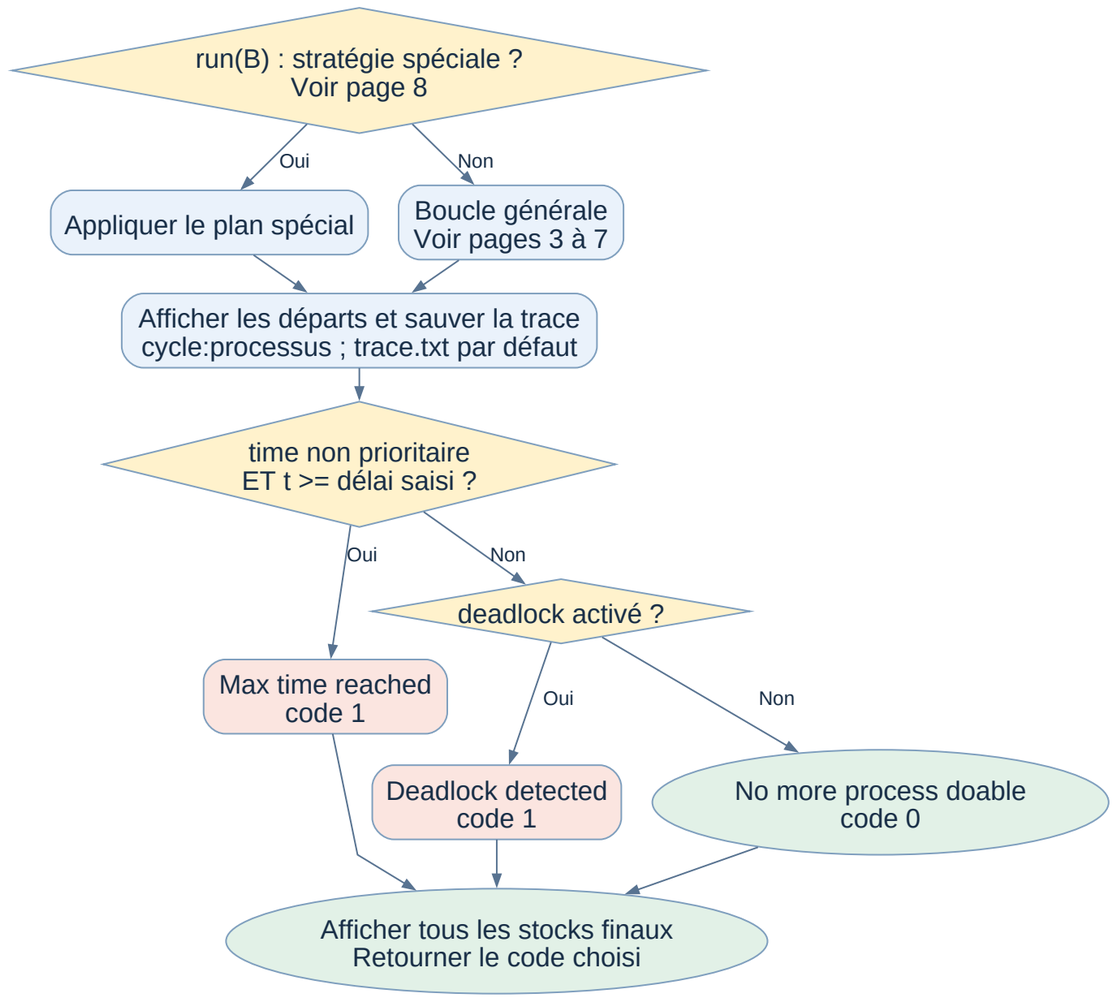
1. De la commande à la trace

Lecture de haut en bas • losange = décision • Oui / Non = branche du code • B = borne effective en cycles

Entrée et choix de la borne



Simulation, sauvegarde et code de sortie



Sources : src/krpsim/cli.py:33-241 ; display.py:53-151 ; simulator.py:53-89, 287-321

Trois notions de temps

delay : argument CLI entier strictement positif. **B / max_time** : borne maximale de la simulation. **time** : mot-clé exact, en minuscules, dans optimize. En première position, il impose B = 500 ; ce n'est pas un délai en secondes.

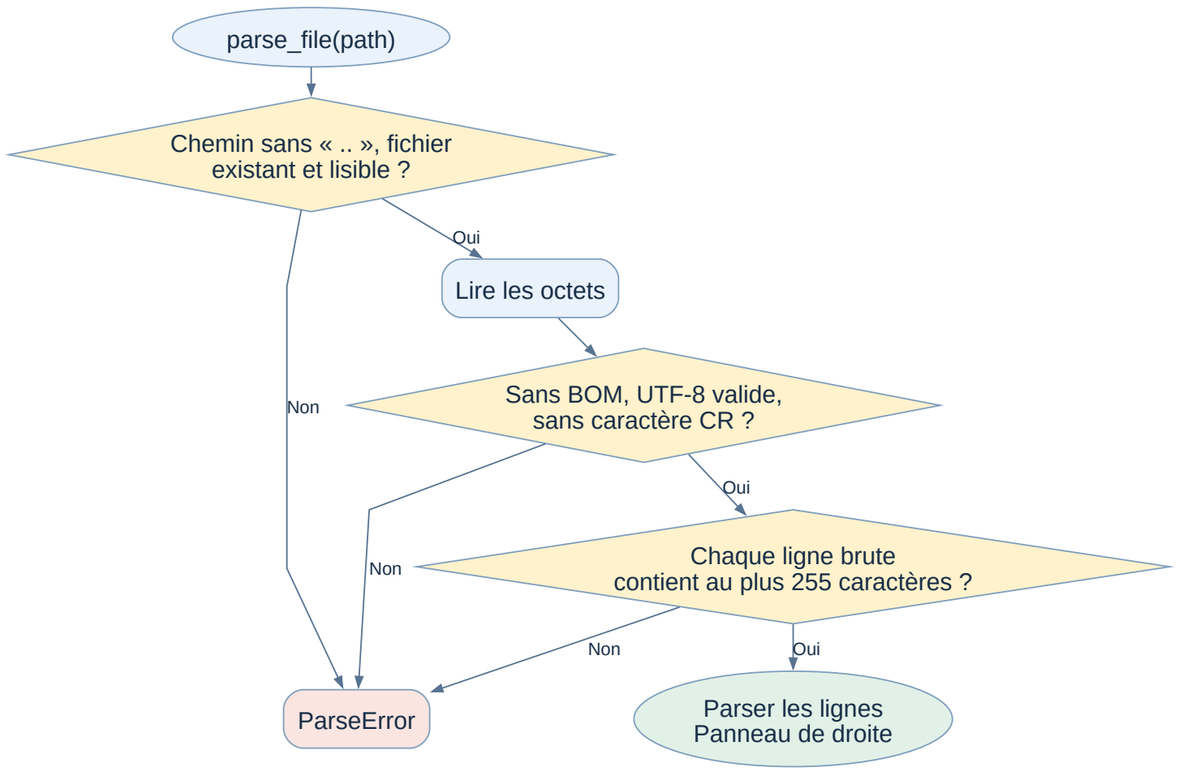
Contrat de la CLI

Chemin sans composant « .. », fichier existant et lisible ; --help retourne 0. La trace est sauvée avant le choix du code de sortie, avec flush et fsync. Une trace vide devient un commentaire. Seules les ParseError sont converties en « invalid config ».

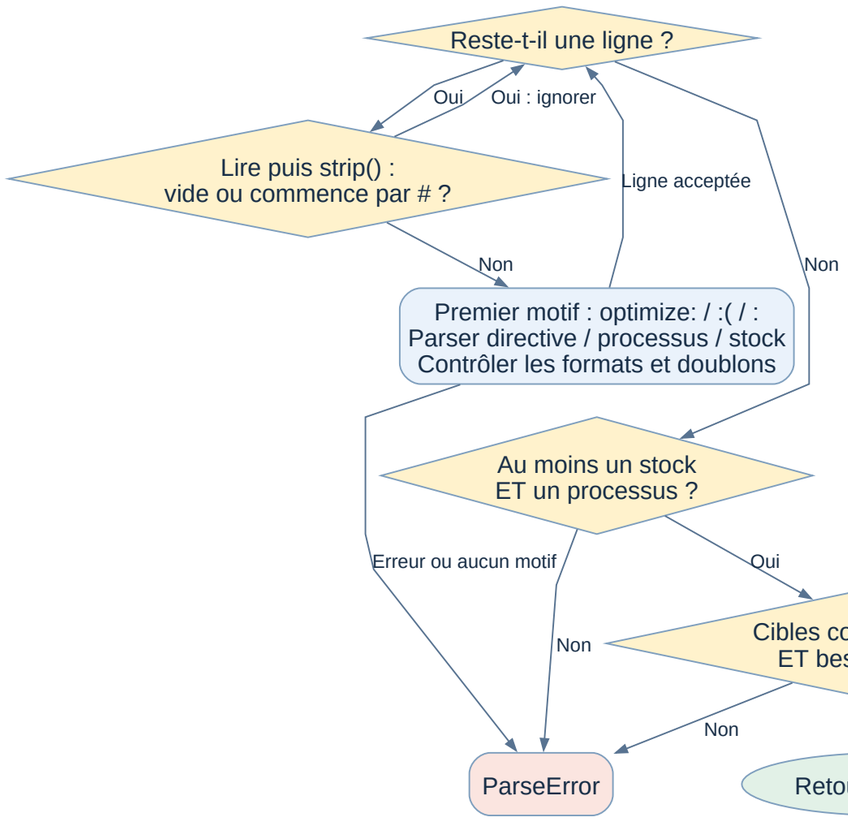
2. Lire et valider la configuration

Le parseur construit Config(stocks, processes, optimize) ; la première ParseError interrompt la lecture

Contrôles du fichier



Interprétation puis validation des références



Sources : src/krpsim/parser.py:109-294, 298-463, 467-619

Grammaire et doublons

Stock : nom non vide, quantité entière >= 0. Processus : nom:(besoins):(résultats):durée, durée entière >= 0 ; quantités des ressources > 0. Blocs vides, résultats omis et éléments vides entre « ; » sont acceptés. Doublons de stocks, processus, ressources ou cibles : rejet.

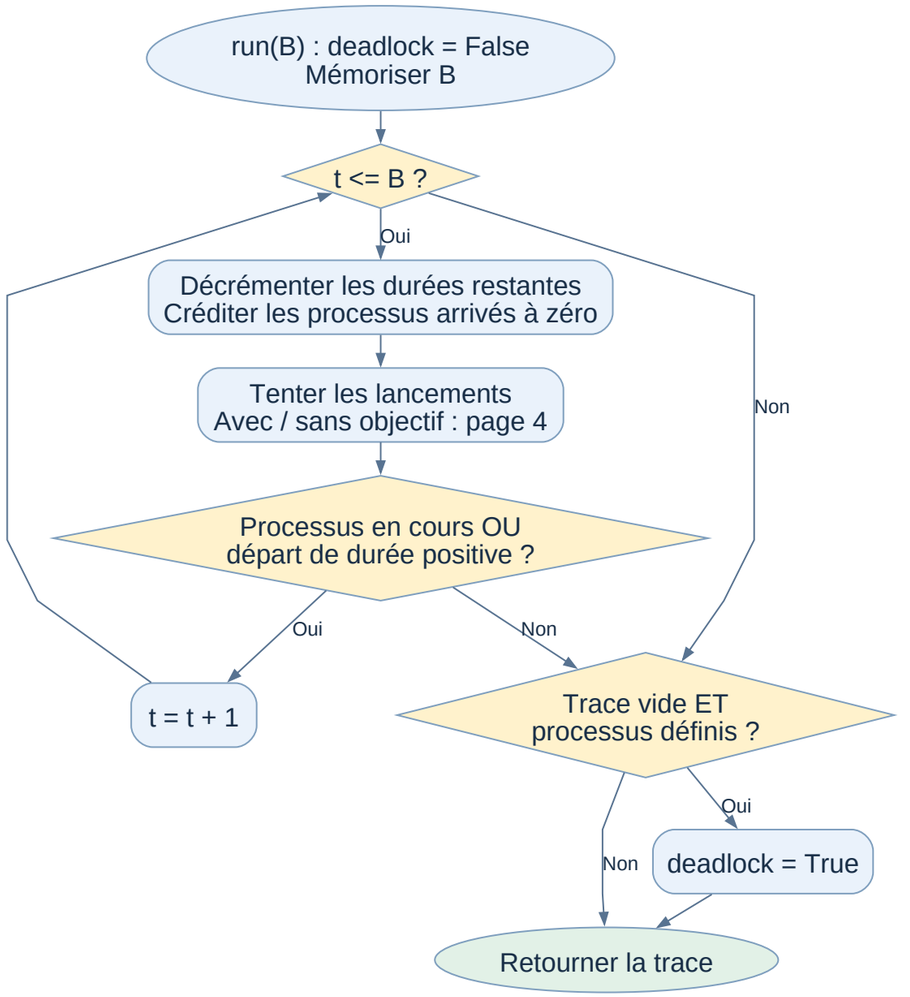
Références et ordre

Une seule directive optimize, facultative et non vide ; ses cibles sont connues dans les stocks, besoins ou résultats, sauf time. Chaque besoin doit figurer dans un stock initial ou un résultat de processus. Aucun ordre de sections n’est imposé. Les cycles de dépendance ne sont pas rejetés ici.

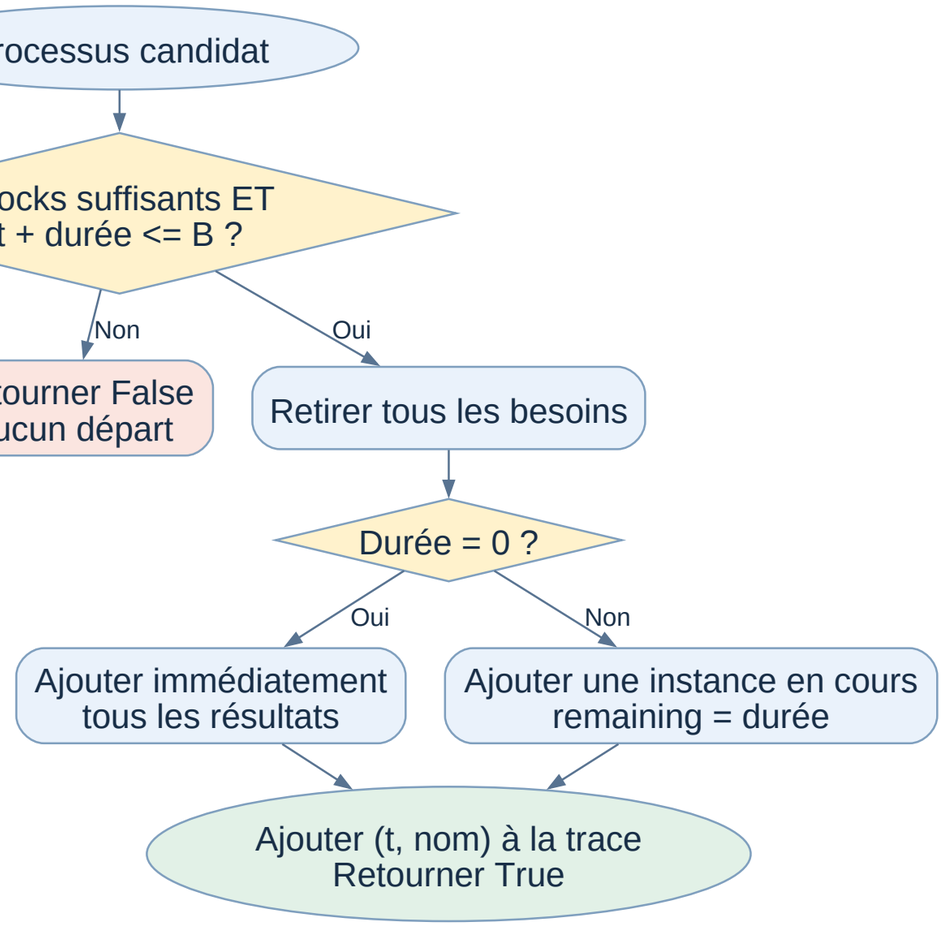
3. Un cycle : terminer, lancer, avancer

Le temps t est le cycle courant ; une durée d se termine au cycle de départ + d

Boucle générale, après refus de la stratégie spéciale



Tentative de lancement : _launch(process)



Sources : src/krpsim/simulator.py:92-173, 257-321 ; cli.py:180-194

Borne inclusive et arrêt

Finir exactement à B est permis par _can_start. Hors time prioritaire, la CLI retourne pourtant 1 dès que t >= délai demandé. Une étape purement instantanée ne fait pas avancer t : la boucle run s’arrête si aucun processus de durée positive n’est en cours ou n’a démarré.

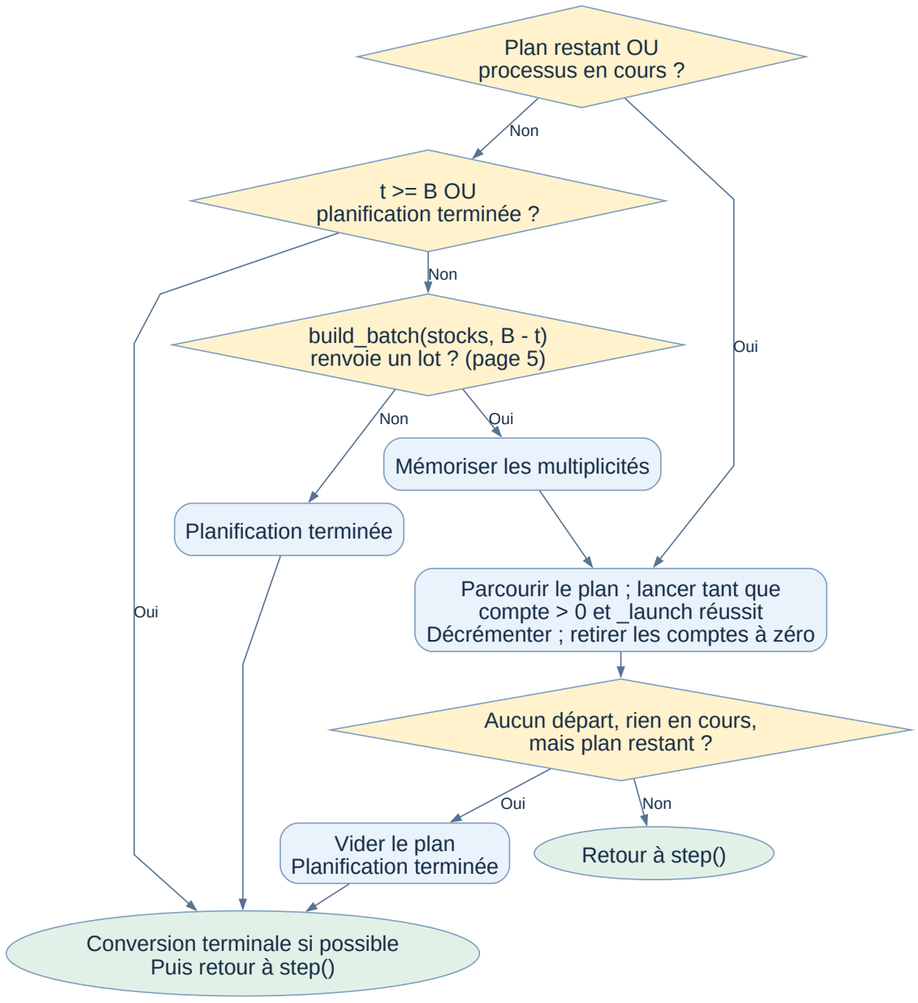
Ce que deadlock signifie ici

Dans la boucle générale, le drapeau signifie uniquement « aucun départ dans toute la trace malgré des processus définis ». Un blocage après des départs ne l’active pas. Le retour anticipé de la stratégie spéciale contourne ce test. Les stocks sont consommés dès le départ, pas à la fin.

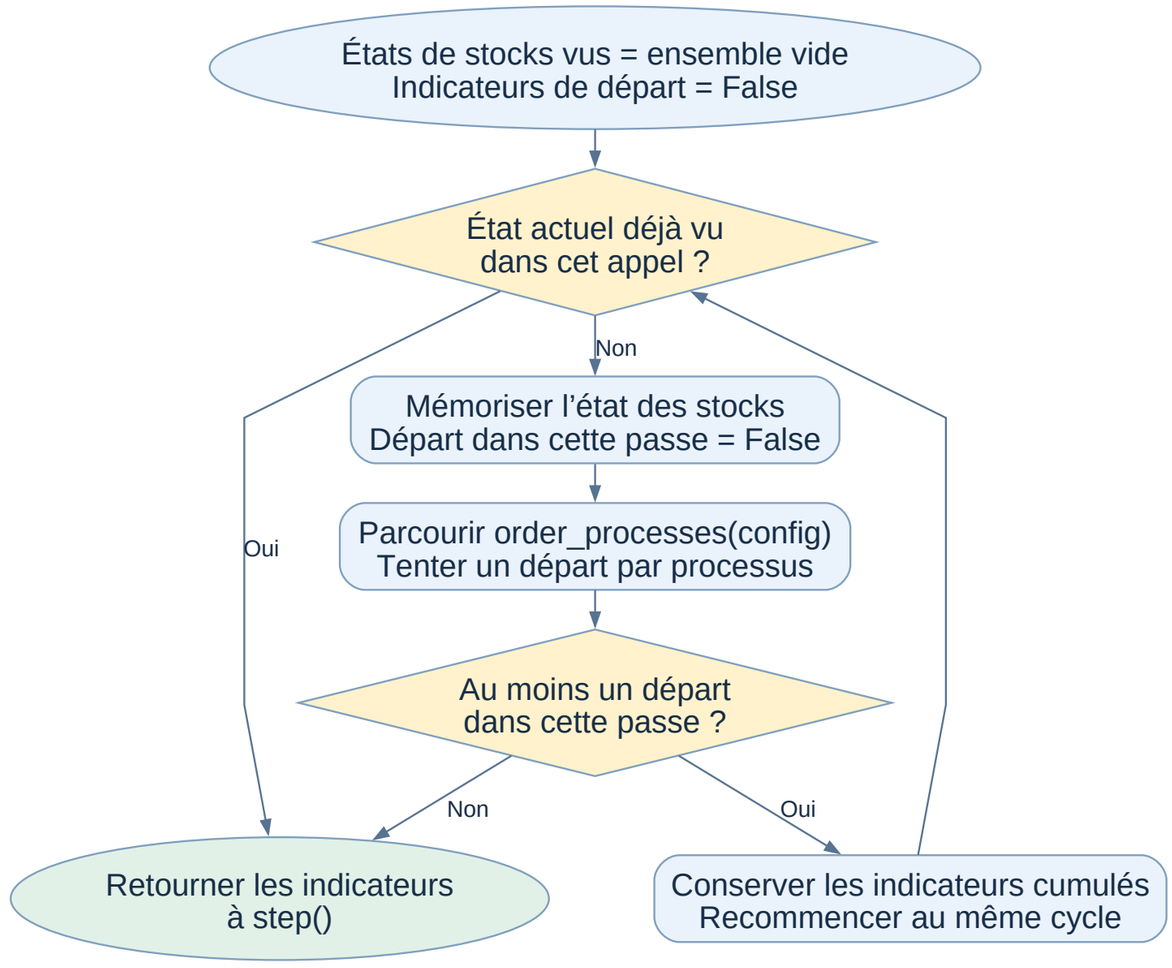
4. Choisir les départs au cycle courant

Objectif actif = première cible hors time, ou ressource intermédiaire d’un convertisseur terminal reconnu

Avec objectif : exécuter le lot préparé



Sans objectif : répéter les passes de lancement



Sources : src/krpsim/simulator.py:131-254 ; optimizer.py:49-97, 372-441

Ordre des processus et répétitions

Sans cible de ressource : ordre alphabétique si optimize est absent ; avec time seul, durée croissante puis nom. Un processus peut repartir au même cycle via plusieurs passes. En mode objectif, l’ordre est celui des clés du plan ; chaque multiplicité est lancée tant que les stocks et B le permettent.

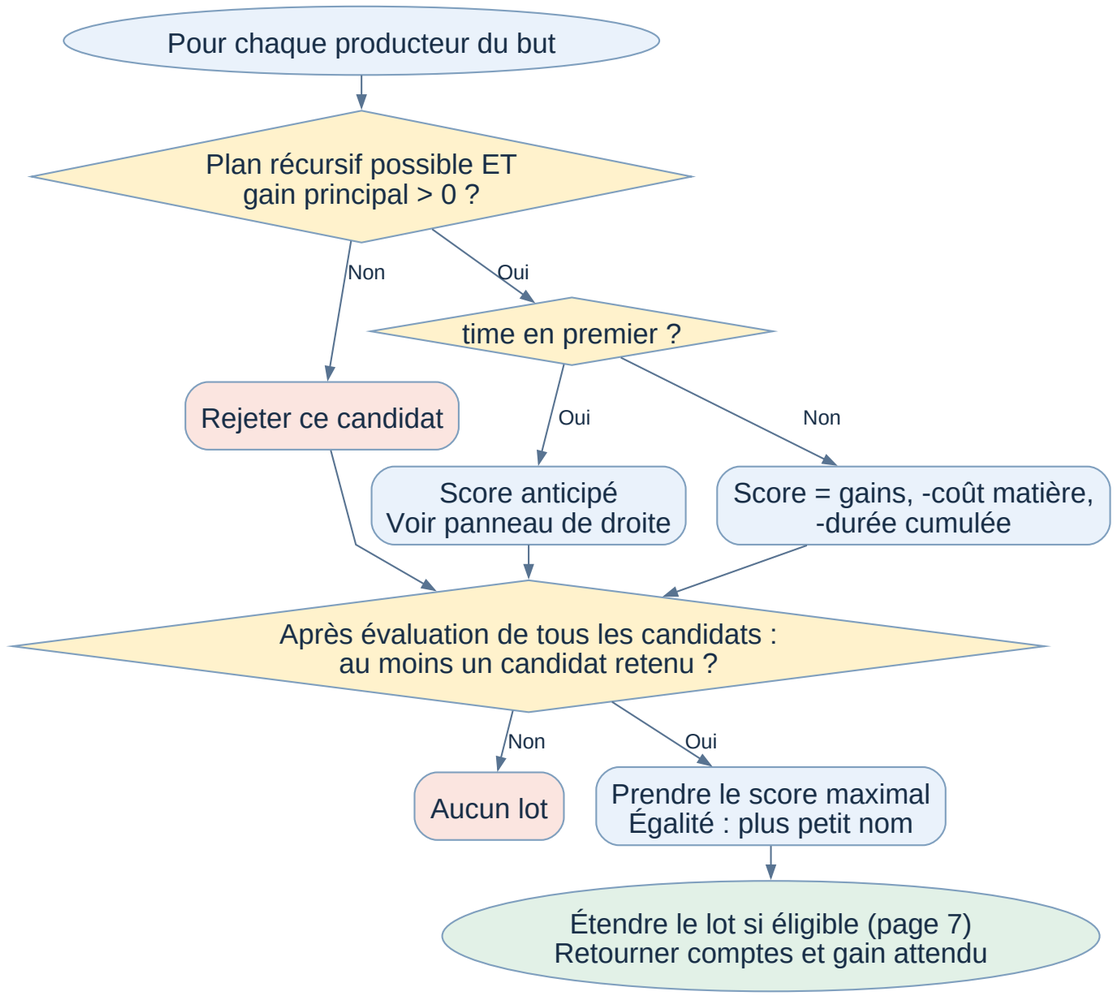
Conversion terminale et limite de protection

La conversion finale consomme immédiatement le maximum du stock convertible quand la planification s’arrête (page 7). Sans objectif, la protection compare seulement les stocks entre passes : elle stoppe un état répété, mais ne borne pas toute croissance instantanée infinie.

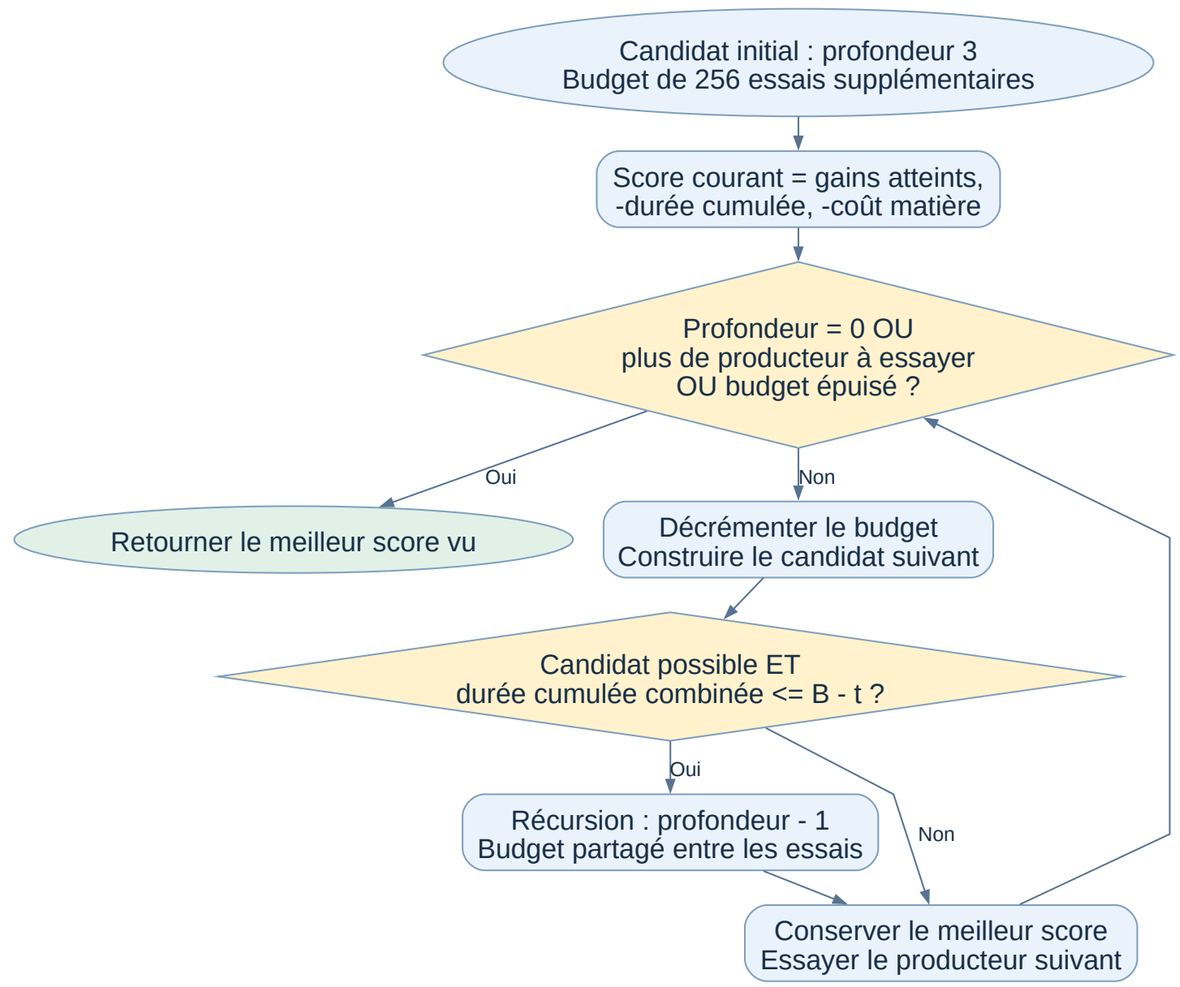
5. Construire le meilleur lot de production

Le planificateur travaille sur des copies des stocks ; il renvoie des nombres de lancements, pas un calendrier

Évaluer et retenir un candidat



Si time est le premier critère : anticiper la suite



Sources : src/krpsim/optimizer.py:46-47, 99-285 ; simulator.py:152-158, 200-212

Comment comparer les scores

Les gains sont comparés dans l’ordre des cibles, sans time. Si un convertisseur terminal est reconnu, le gain porte sur son entrée. La durée est somme(durée × nombre de départs), pas le temps réel avec parallélisme. Le coût compte les besoins des ressources que chaque processus restitue en quantité insuffisante.

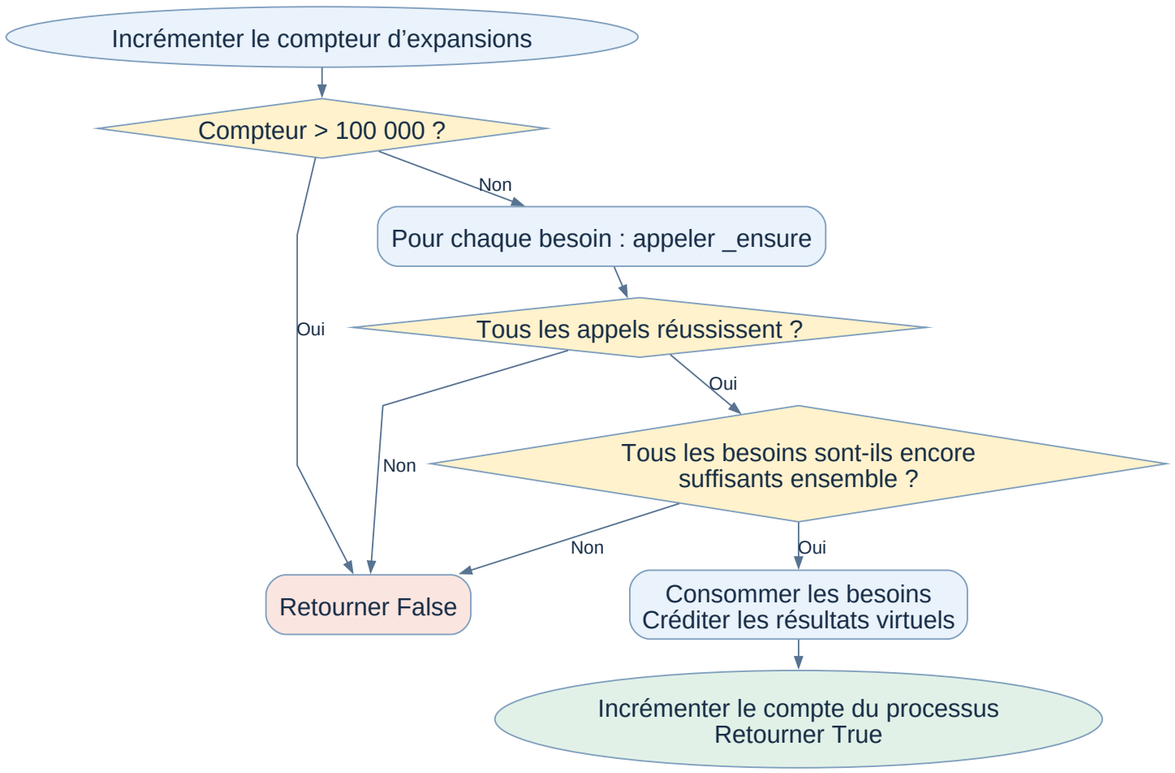
Portée exacte de l’anticipation

Jusqu’à trois lots supplémentaires, avec 256 essais par candidat initial. La borne filtre les prolongements combinés ; elle ne rejette pas directement tout lot initial ou étendu trop long. Les lancements réels restent filtrés par t + durée <= B. Le lot renvoyé reste le lot initial, éventuellement étendu ; l’anticipation sert au classement.

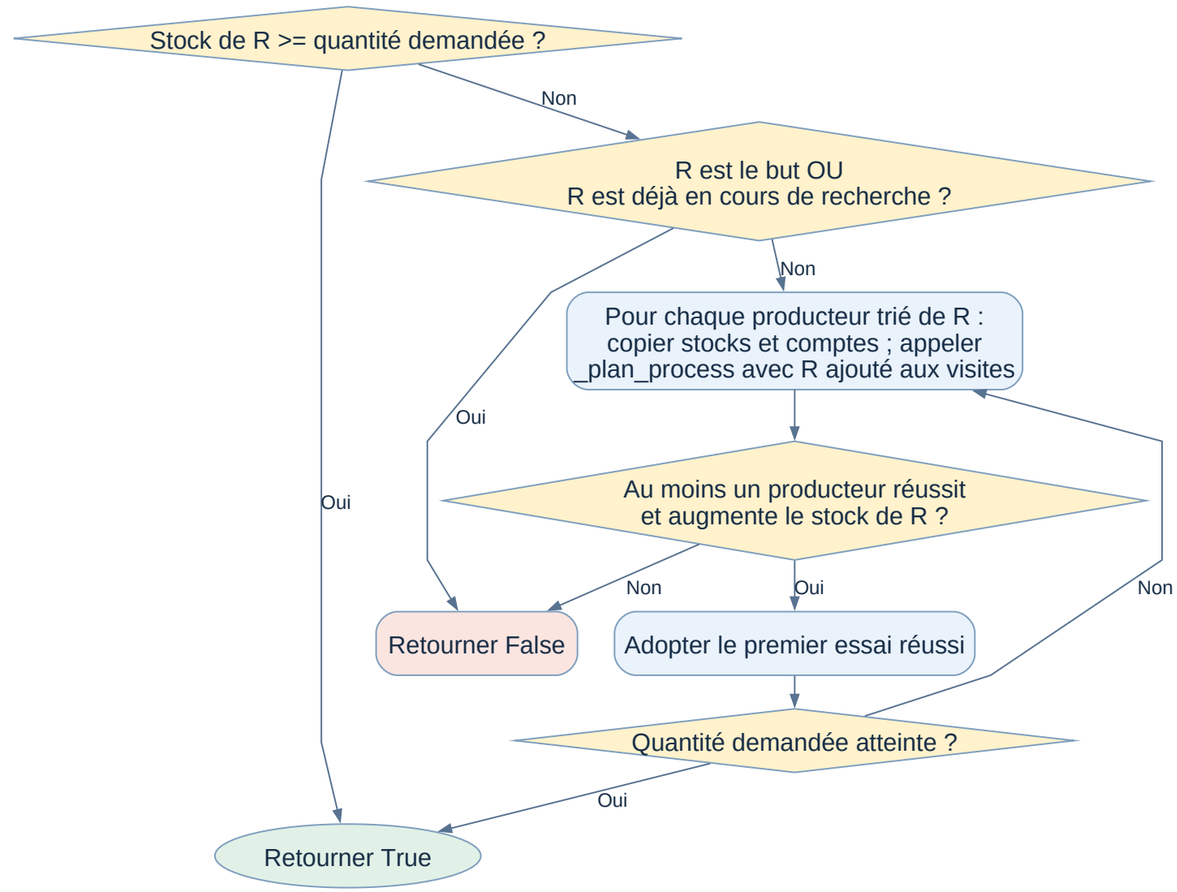
6. Préparer les dépendances sur les stocks virtuels

Chaque essai de producteur utilise une copie ; seules les tentatives réussies sont adoptées

_plan_process : préparer puis appliquer une exécution



_ensure : obtenir une quantité de ressource



Sources : src/krpsim/optimizer.py:287-369 (_ranked_producers, _ensure, _plan_process)

Tri des producteurs intermédiaires

Priorité : gain net de la ressource décroissant, quantité produite décroissante, durée croissante, somme des besoins croissante, puis nom alphabétique. L’essai doit augmenter la ressource recherchée. Si aucun producteur n’y parvient, _ensure échoue.

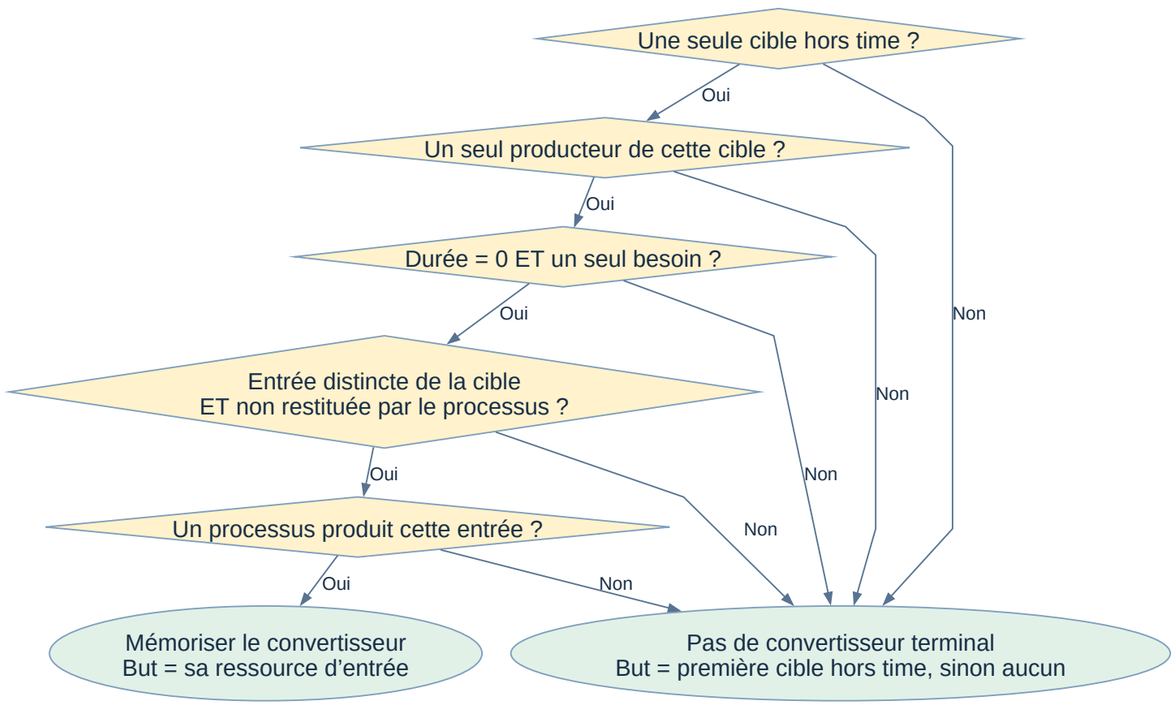
Contrôle collectif des besoins

Une dépendance peut consommer une ressource déjà préparée pour un autre besoin. Le code revérifie donc tous les besoins ensemble avant de les débiter. Les cycles refusés et la recherche bornée ne prouvent pas qu’aucune autre solution globale existe.

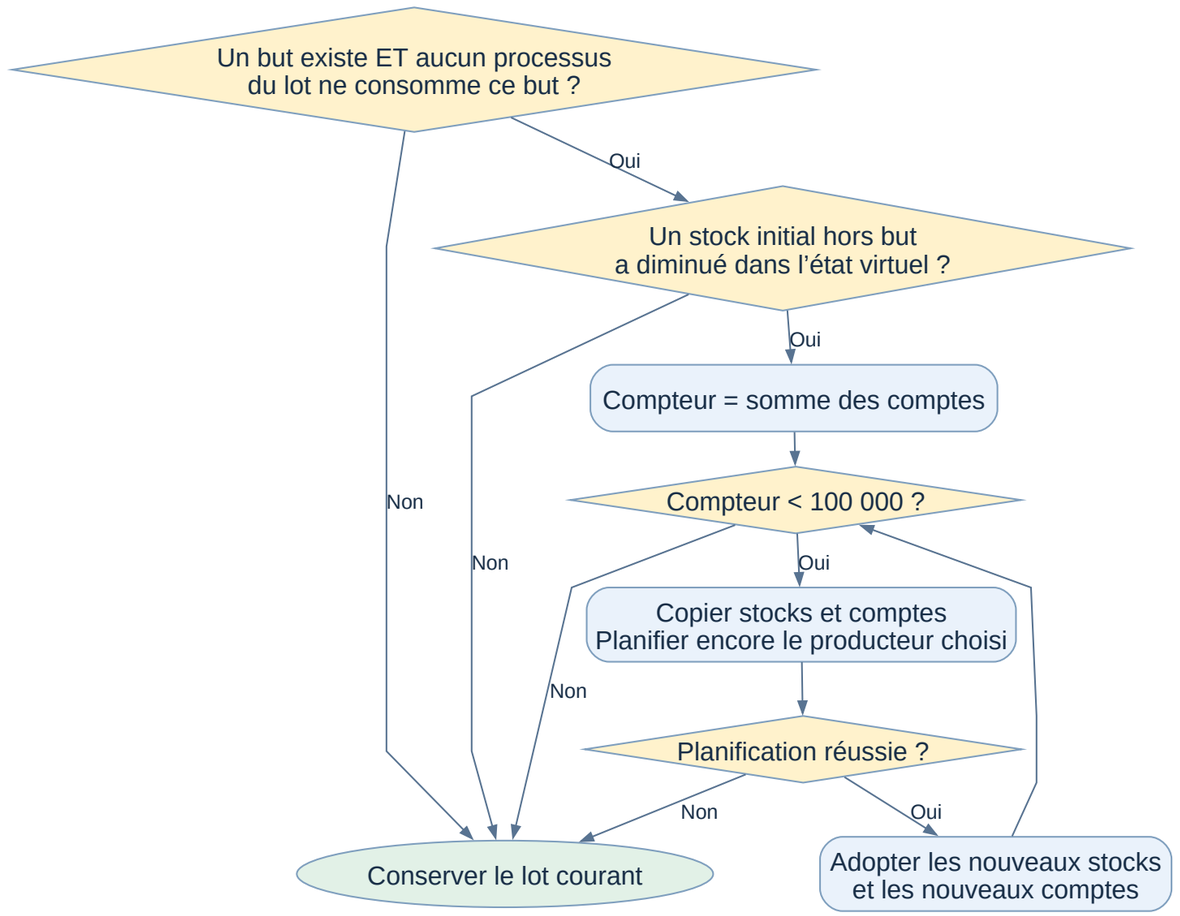
7. Répéter un lot et différer la conversion finale

Deux adaptations distinctes : conserver le fonds de roulement et regrouper des répétitions utiles

Reconnaître un convertisseur terminal à l’initialisation



Étendre le lot retenu par build_batch



Sources : src/krpsim/optimizer.py:49-97, 244-275 ; simulator.py:175-212

Exécuter la conversion terminale

Quand la préparation s’arrête ou qu’un plan reste bloqué, _launch_terminal calcule stock_entrée // besoin_entrée. S’il existe un convertisseur et que ce nombre est positif, il appelle _launch autant de fois ; sinon, il retourne sans départ. La ressource intermédiaire finance ainsi les lots précédents.

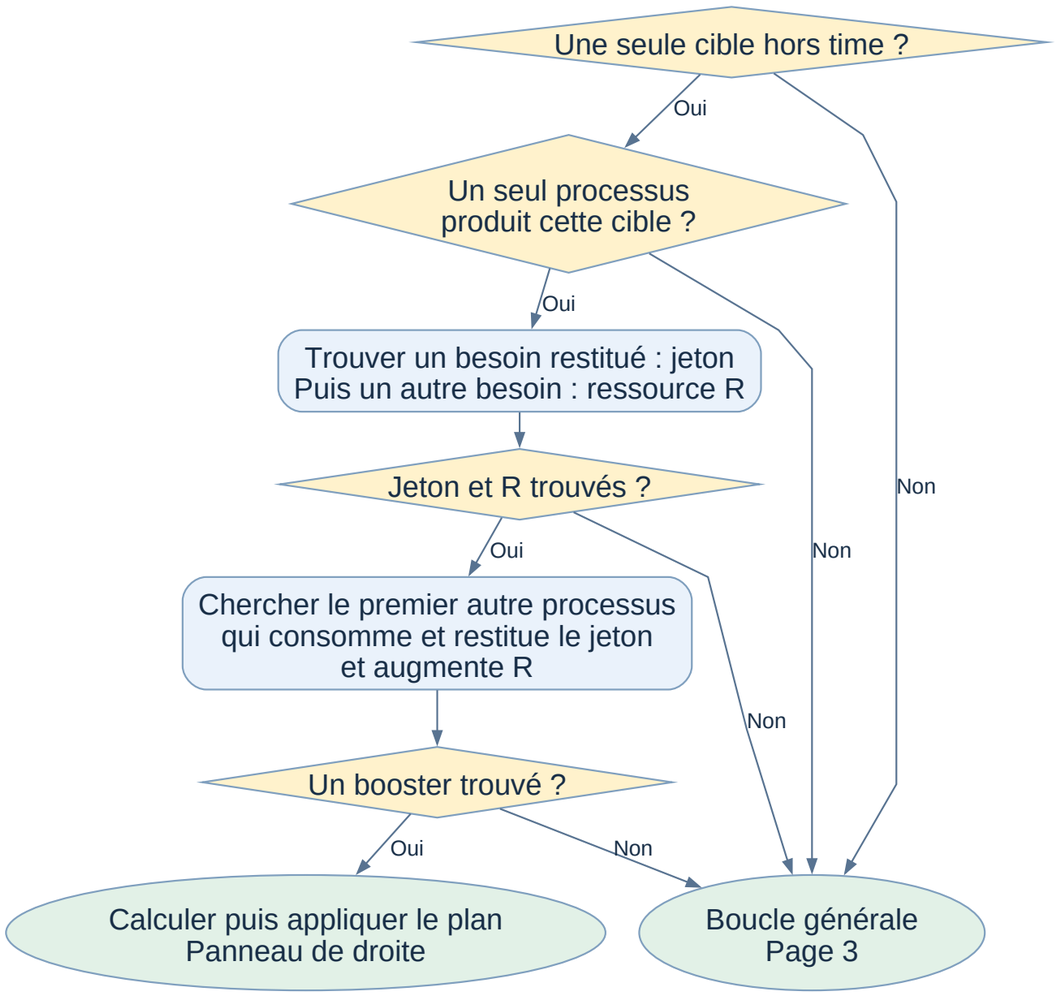
Rôle et portée de l’extension

Le lot est prolongé sur des stocks virtuels jusqu’au premier échec ou au plafond d’expansions. Le compteur de l’extension commence à la somme des multiplicités. Cette étape n’utilise pas directement B ; les départs réels restent soumis au contrôle de la page 3.

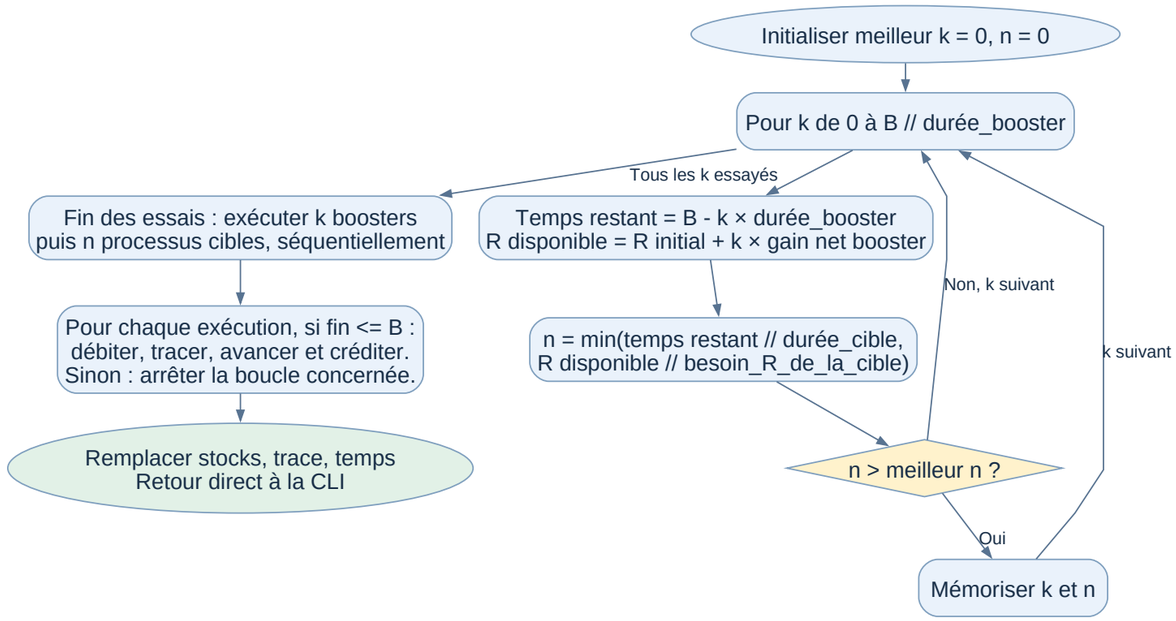
8. Le raccourci « booster puis cible »

Ce chemin est testé avant la boucle générale et applique directement une exécution séquentielle

Reconnaître une topologie compatible



Choisir le nombre de boosters puis appliquer le plan



Sources : src/krpsim/simulator.py:324-641 (_single_target, _target_process, _split_resources, _find_booster, _best_loops, _apply_custom_plan)

Sélection exacte

Le jeton est le premier besoin restitué au moins intégralement ; R est le premier autre besoin. Le code n’exige pas exactement deux besoins. Le premier booster compatible dans l’ordre de configuration est retenu. En cas d’égalité de production, le premier k est conservé.

Hypothèses non vérifiées par ce raccourci

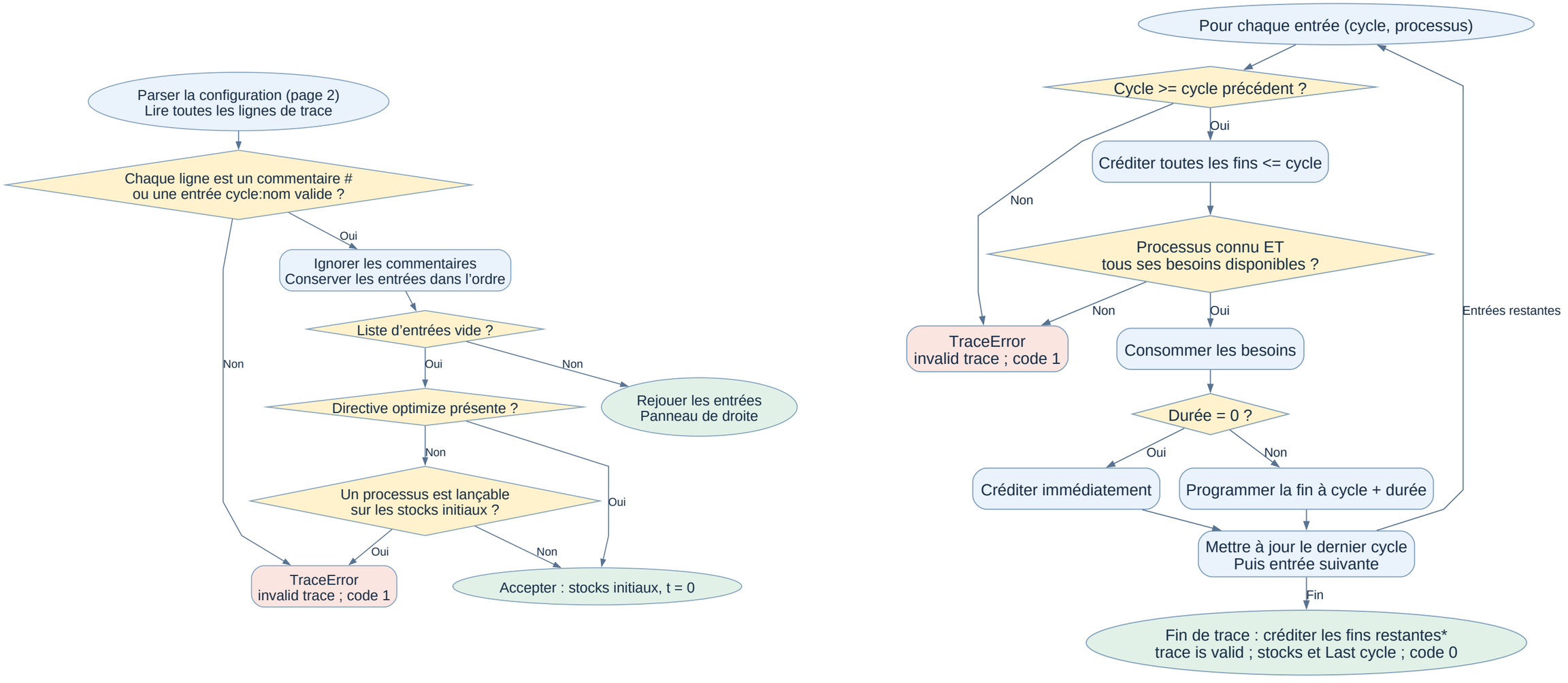
Les divisions supposent des durées non nulles. L’application directe ne réutilise pas _can_start : la disponibilité de tous les besoins n’est pas contrôlée. Le calcul ne modélise qu’une ressource R et le temps. Le retour anticipé laisse deadlock à False, même avec une trace vide.

9. Vérifier une trace par rejeu

krpsim_verif <config> <trace> • le vérificateur accepte tout ordonnancement réalisable, même différent du simulateur

Lire la configuration et la trace

Rejouer les départs dans l'ordre du fichier



Sources : src/krpsim_verif/verifier.py:63-280 ; src/krpsim_verif/cli.py:25-102

Ce que valide le vérificateur

Format : cycle numérique, nom non vide ; une ligne vide est une erreur, un fichier vide est permis. Aucun strip des espaces ; # doit être en début de ligne. Le rejeu contrôle chronologie et ressources, sans exiger l'optimalité, l'achèvement du but ou le délai CLI. Répétitions au même cycle et traces partielles sont acceptées.

Sorties et limite de finalisation

ParseError donne « invalid config » ; OSError ou TraceError donne « invalid trace », code 1. Les erreurs argparse donnent 2. Les autres exceptions ne sont pas converties par cette CLI. *La finalisation crédite les fins <= 2^63 - 1 ; ce plafond est fixé dans le code.

10. Exemples exécutés et commandes actuelles

Les résultats ci-dessous sont calculés avec le code local puis contrôlés par le vérificateur

IKEA : les composants partent ensemble

Commande : krpsim resources/ikea 10
optimize:(time;armoire) → B = 500

Cycle	Départ	Nombre
0	do_montant	2
0	do_fond	1
0	do_etagere	3
20	do_armoire_ikea	1

Cycle final : 50. Armoire = 1 ; tous les autres stocks = 0.
7 lignes de trace ; CLI = 0 ; vérification = 0.

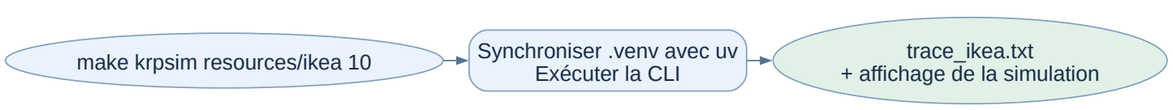
STEAK : l’anticipation conserve la production complète

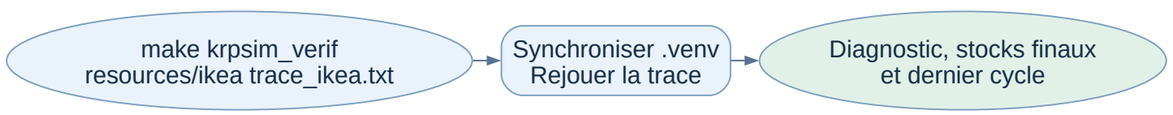
Commande : krpsim resources/steak 10
optimize:(time;steak_cuit) → B = 500

Cycle	Départ	Effet à la fin
0	cuisson_1	2 mi-cuits au cycle 10
10	cuisson_3	1 cuit au cycle 20
20	cuisson_2	2 cuits au cycle 30

Cycle final : 30. Cuits = 3 ; crus = 0 ; mi-cuits = 0 ; poêle = 1.
3 lignes de trace ; CLI = 0 ; vérification = 0.

Flux Make actuel





Sources : resources/ikea ; resources/steak ; Makefile:55-94, 156-266 ; tests/test_evaluation.py:13-48

Traces texte uniquement

La CLI directe écrit trace.txt, sauf --trace. La cible make krpsim choisit trace_<nom_sans_extension>.txt. Le vérificateur affiche son résultat. La génération de JSON graphique et de diagrammes de Gantt a été supprimée du flux actuel.

Codes des commandes Make

Les codes 0 / 1 / 2 des pages précédentes sont ceux des CLI Python. Les recettes Make affichent les erreurs puis terminent normalement ; elles ne propagent généralement pas l’échec de la CLI. Une erreur de synchronisation de l’environnement peut toutefois faire échouer Make.

11. Périmètre, références et points de lecture

Document descriptif de l’implémentation locale ; les limites observées sont conservées dans les arbres

Tous les modules applicatifs relus

Fichier	Rôle dans le document	Empreinte SHA-256 (12 premiers caractères)
src/krpsim/__init__.py	Version du paquet via importlib.metadata ; hors décisions métier	0fdeea3ce2c7
src/krpsim/cli.py	Arguments, borne effective, erreurs et sorties - page 1	43fe095630de
src/krpsim/parser.py	Format, stocks, processus et références - page 2	c2f56a15c955
src/krpsim/display.py	En-tête, format de trace, écriture et synchronisation - page 1	b104649e6dcc
src/krpsim/simulator.py	Cycles, lancements, lots et raccourci spécial - pages 3, 4 et 8	d31f29f85efb
src/krpsim/optimizer.py	Objectifs, scores, dépendances et conversion - pages 4 à 7	c3a99f680d36
src/krpsim_verif/__init__.py	Déclaration du paquet ; aucune décision d’exécution	26dd07d9f140
src/krpsim_verif/cli.py	Arguments et diagnostics de vérification - page 9	f40715a16b9f
src/krpsim_verif/verifier.py	Parsing et rejeu des traces - page 9	b85b1032ef08

Vérifications de cette édition

Lecture complémentaire : Makefile, pyproject.toml, workflows CI et tests. Exemples IKEA et STEAK exécutés avec les CLI puis rejoués. Suite pytest sous Python 3.10.20 (version CI) : **163 tests réussis ; couverture des instructions et des branches à 100 %**.

Sources : Ensemble de src/ ; Makefile ; pyproject.toml ; .github/workflows/ci.yml ; tests/

Arbres de contrôle, pas preuve d’optimalité

Les flèches de retour représentent des boucles du programme. Les sorties d’erreur nommées couvrent les branches explicites ; une exception technique non interceptée peut interrompre le parcours. Le planificateur est heuristique et borné : aucune optimalité globale n’est affirmée.

Version locale traçable

Les numéros de lignes correspondent aux fichiers lus pour cette édition. L’empreinte du pied de page couvre les 9 modules Python, Makefile et pyproject.toml. HEAD seul est insuffisant, car le dépôt contient des modifications locales. Le code applicatif n’a pas été modifié pour produire ce document.